UNIVERSITÉ MUNDIAPOLIS — ÉCOLE D'INGÉNIEURS

3ème Année CI — Génie Logiciel

Prise en main de Burp Suite

Intercepter et modifier du trafic HTTP

Module : Sécurité des SI et Cybersécurité

Travaux Pratiques 01 — L'Arsenal de l'Auditeur

Réalisé par · Fellah Youssef
Encadré par · Prof. Khoudali Saad
Cible · https://ginandjuice.shop

Année Universitaire : 2026–2027

Table des matières

Introduction **1**

1 Exercice 1 : Observer le trafic HTTP **1**
1.1 Ligne de requête (*Request Line*) . . . 2
1.2 Identification et rôle des en-têtes (*Headers*) . . . 2
1.3 Code de statut de la réponse HTTP . . . 3
1.4 Volume de requêtes pour la page d'accueil et réalité du web moderne . . . 3

2 Exercice 2 : Intercepter et modifier une requête HTTP **4**
2.1 Produit cliqué vs produit affiché . . . 4
2.2 Décisionnaire final de l'affichage . . . 5
2.3 Test d'un identifiant inexistant . . . 5

3 Exercice 3 : Repeater — Rejouer et tester sans intermédiaire **6**
3.1 Tableau récapitulatif des tests sur le paramètre `productId` . . . 7
3.2 Comportement face à un `User-Agent` fantaisiste et niveau de confiance . . . 7
3.3 Exploitation du rejeu de requêtes par un attaquant . . . 7

4 Exercice 4 : Intercepter et modifier une réponse HTTP **8**
4.1 Affichage obtenu dans le navigateur . . . 8
4.2 Visibilité de la modification pour le serveur et le navigateur . . . 8
4.3 Scénario d'attaque de type *Man-in-the-Middle* (MitM) . . . 9

5 Questions de synthèse **9**
5.1 Interception HTTPS et modèle de confiance cryptographique . . . 9
5.2 Trois actions concrètes réalisables par un attaquant armé d'un proxy . . . 10
5.3 Réfutation de l'argument : « De toute façon ce champ est en lecture seule dans le formulaire, personne ne peut le modifier » . . . 10
5.4 Impact direct sur les habitudes de développement logiciel . . . 10

Conclusion **11**

Introduction

Ce compte rendu présente la prise en main de **Burp Suite** (édition Community) dans le cadre du module *Sécurité des SI et Cybersécurité*, session 2 : *L'Arsenal de l'Auditeur (HTTP et Proxys)*.

Auteur : Youssef Fellah
Formation : 3ACI INFO GL
Cible d'évaluation : https://ginandjuice.shop

La cible est une application web volontairement vulnérable, conçue pour tester les scanners de vulnérabilités. Le TP comporte quatre exercices (observation du trafic, interception d'une requête, rejeu avec Repeater, interception d'une réponse) suivis de questions de synthèse.

1 Exercice 1 : Observer le trafic HTTP

L'objectif de cet exercice est d'analyser le trafic réseau généré lors de la navigation sur une application web via le proxy d'interception Burp Suite, en examinant la structure exacte des requêtes et réponses HTTP.

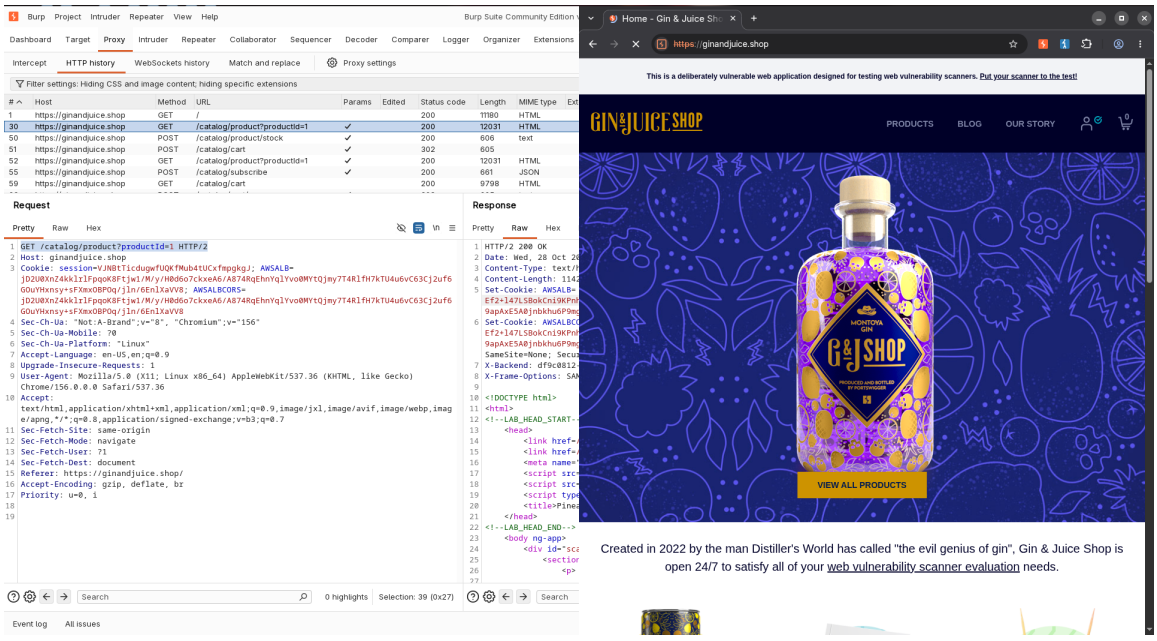


Figure 1: Vue de la boutique et historique HTTP dans Burp Suite.

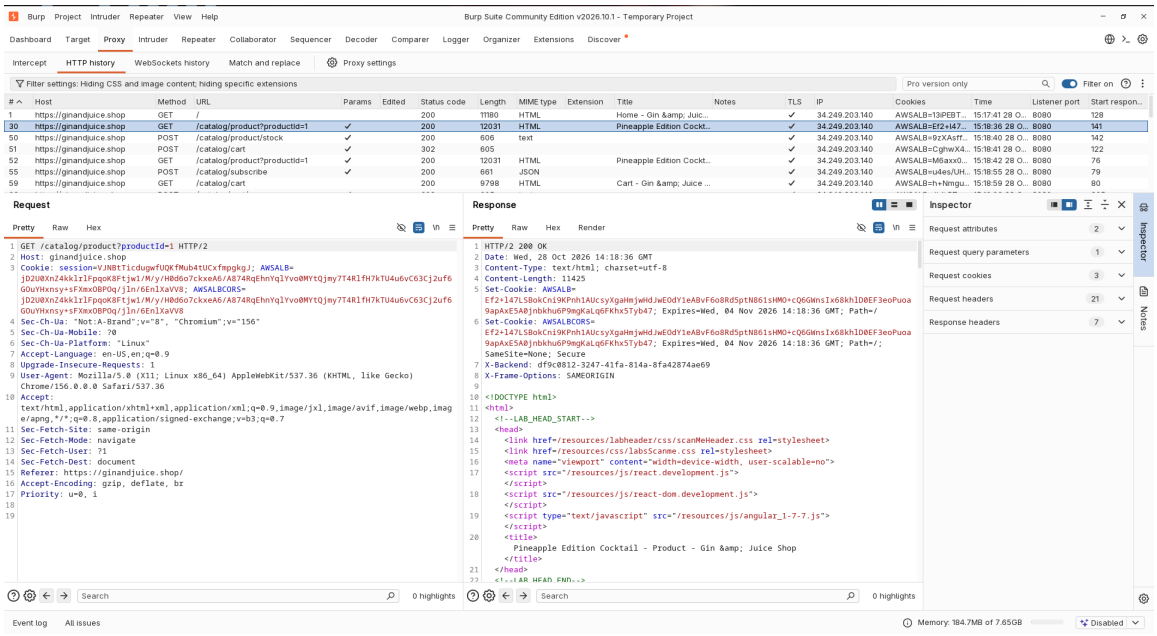


Figure 2: Inspection de la requête GET et de la réponse associée.

1.1 Ligne de requête (*Request Line*)

Pour la consultation de la fiche du premier produit :

```
GET /catalog/product?productId=1 HTTP/2
```

Décomposition technique des composants :

- **Méthode (verbe HTTP) :** `GET` — indique une demande de récupération d'une ressource sans modification de l'état du serveur (méthode sûre et idempotente selon la norme HTTP).
- **Chemin cible (URI / *query string*) :** `/catalog/product?productId=1` — chemin relatif de l'*endpoint* d'affichage de produit, complété par le paramètre de requête `productId=1` ciblant la clé d'identification du produit.
- **Version du protocole :** `HTTP/2` — protocole de transmission binaire, multiplexé et compressé (HPACK), transporté sur une session TLS.

1.2 Identification et rôle des en-têtes (*Headers*)

Valeurs extraites de la requête interceptée :

```
Host: ginandjuice.shop
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like
    Gecko) Chrome/156.0.0.0 Safari/537.36
Cookie: session=VJNBtTicdugwfUQKfMub4tUCxfmpgkgJ; AWSALB=
    jD2U0XnZ4kklrlFpqoK8Ftjw1/M/y/H0d6o7ckxeA6/
    A874RqEhnYqlYvo0MYtQjmy7T4RlfH7kTU4u6vC63Cj2uf6GOuYHxnsy+sFXmxOBPOq/jln/6
    EnlXaVV8; AWSALBCORS=jD2U0XnZ4kklrlFpqoK8Ftjw1/M/y/H0d6o7ckxeA6/
    A874RqEhnYqlYvo0MYtQjmy7T4RlfH7kTU4u6vC63Cj2uf6GOuYHxnsy+sFXmxOBPOq/jln/6
    EnlXaVV8
```

Rôle de chaque en-tête :

- **Host :** spécifie le nom de domaine complet ciblé par la requête. Cet en-tête est obligatoire en HTTP/1.1 et standard en HTTP/2 pour permettre au serveur frontal d'acheminer le trafic vers l'hôte virtuel approprié (*virtual hosting*) lorsqu'une même adresse IP héberge plusieurs applications web distinctes.

- **User-Agent :** décrit la configuration logicielle et matérielle du client émetteur (navigateur Chromium version 156, moteur de rendu Blink/WebKit, exécuté sur un noyau Linux 64 bits). Il permet au serveur de journaliser l'environnement client et, le cas échéant, d'adapter dynamiquement la disposition de l'interface utilisateur.
- **Cookie :** transmet des paires clé-valeur persistantes stockées par le navigateur pour maintenir l'état d'une session sur un protocole HTTP fondamentalement sans état (*stateless*) :
  - `session` : jeton de session applicative permettant au serveur d'associer les requêtes au panier d'achat et aux actions de l'utilisateur.
  - `AWSALB` et `AWSALBCORS` : cookies techniques injectés par l'*Application Load Balancer* d'AWS assurant la persistance de session (*session affinity / sticky session*) afin de router les requêtes successives vers le même nœud serveur applicatif.

1.3 Code de statut de la réponse HTTP

```
HTTP/2 200 OK
```

- **Signification :** le code de statut `200 OK` atteste que la requête a été reçue, comprise et traitée avec succès par le serveur.
- **Contenu délivré :** le serveur a localisé la ressource correspondante (`productId=1`) et renvoie dans le corps de sa réponse le document HTML structuré de la fiche produit (`Content-Type: text/html; charset=utf-8`, d'une taille de 11 425 octets).

1.4 Volume de requêtes pour la page d'accueil et réalité du web moderne

- **Constat :** l'affichage de la page d'accueil ne se résume pas à une unique requête. Dans l'historique HTTP de Burp, la requête initiale sur la racine (`GET /`) porte l'index `#1`, tandis que la première consultation de produit porte l'index `#30`. Près d'une trentaine de requêtes intermédiaires ont été enregistrées, et ce malgré l'application d'un filtre excluant par défaut les images et les feuilles de style CSS.
- **Ordre de grandeur :** le chargement complet d'une page d'accueil e-commerce moderne nécessite couramment entre **40 et 80 requêtes HTTP** distinctes.
- **Enseignement sur le web moderne :** une page web n'est plus un document texte monolithique mais une application distribuée complexe. Le fichier HTML initial ne constitue qu'un squelette structurel. Dès sa réception, le moteur de rendu du navigateur analyse le document et émet une série de requêtes réseau parallèles pour acquérir les dépendances nécessaires au fonctionnement du site :
  1. feuilles de style graphiques (CSS) ;
  2. scripts d'exécution frontend (JavaScript, *frameworks* React et AngularJS observés sur le site) ;
  3. fichiers multimédias et éléments graphiques (images de produits, logos, favicons) ;
  4. polices de caractères distantes (formats WOFF / WOFF2) ;
  5. appels d'API asynchrones (AJAX / Fetch) pour initialiser l'état du panier d'achat et les préférences client.

2 Exercice 2 : Intercepter et modifier une requête HTTP

Cet exercice illustre la rupture de confiance entre le client et le serveur : le proxy intercepte la requête avant son expédition sur le réseau public, autorisant la modification directe des paramètres applicatifs.

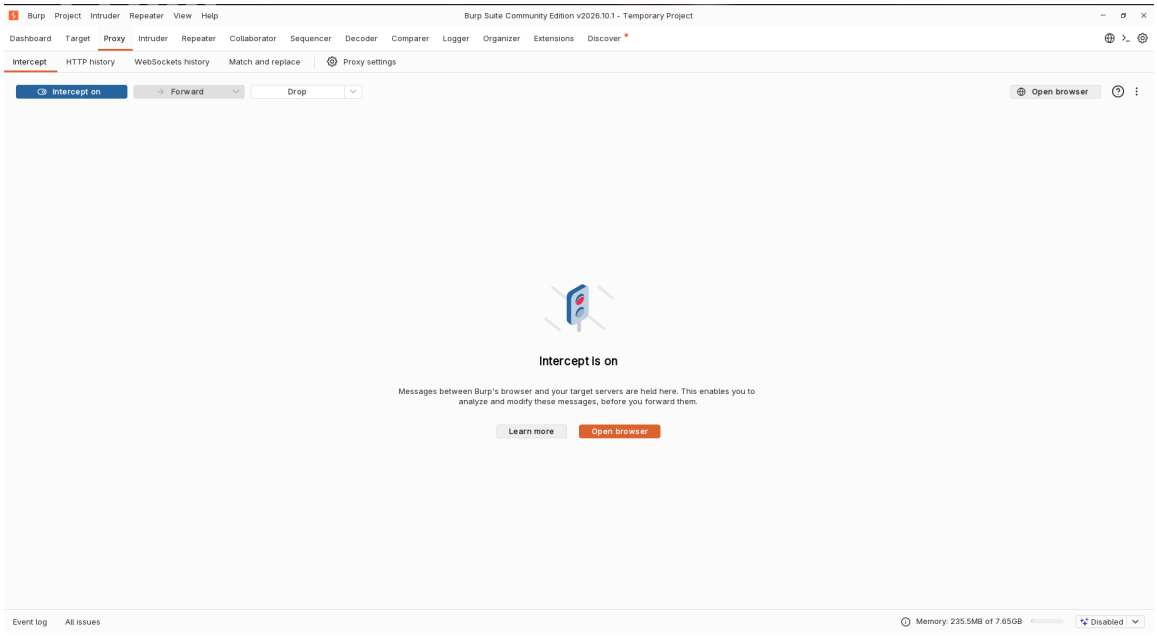


Figure 3: Interception activée dans le sous-onglet *Proxy* → *Intercept*.

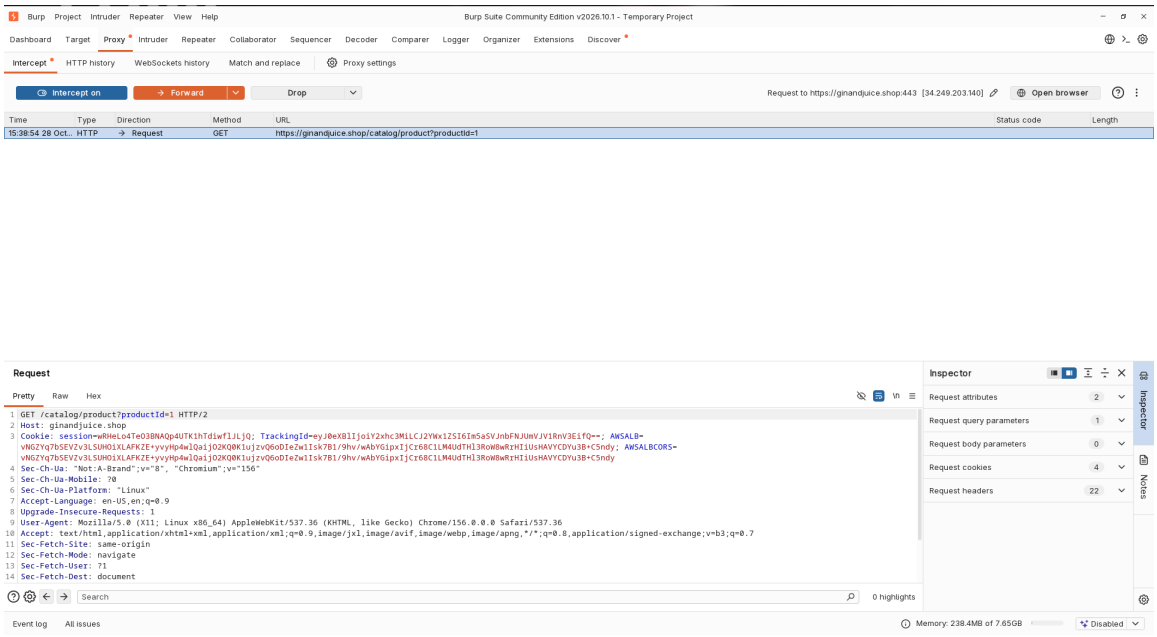


Figure 4: Interception de la requête `GET /catalog/product?productId=1` avant altération du paramètre `productId`.

2.1 Produit cliqué vs produit affiché

- **Action utilisateur dans l'interface :** sélection du produit *Pineapple Edition Cocktail* (`productId=1`).

- **Modification opérée dans l'intercepteur Burp :** remplacement manuel du paramètre dans l'URI avant transmission : `productId=7`.
- **Résultat affiché dans le navigateur :** le navigateur affiche le produit *Fruit Slicer (Limited Edition)*, qui correspond à l'identifiant `7`.

2.2 Décisionnaire final de l'affichage

- **Entité décisionnaire : le serveur.**
- **Explication technique :** le serveur web est totalement agnostique des interactions physiques de l'utilisateur sur son écran (clics de souris, éléments de navigation graphique). Il traite exclusivement le flux de la requête HTTP entrante telle qu'elle se présente au niveau de sa socket réseau. Ayant reçu `productId=7`, son contrôleur applicatif exécute la requête correspondante en base de données, génère le code HTML du produit 7 et le renvoie au client. Le navigateur n'exécute aucune validation logique : il agit en simple moteur de rendu passif des données reçues.

2.3 Test d'un identifiant inexistant

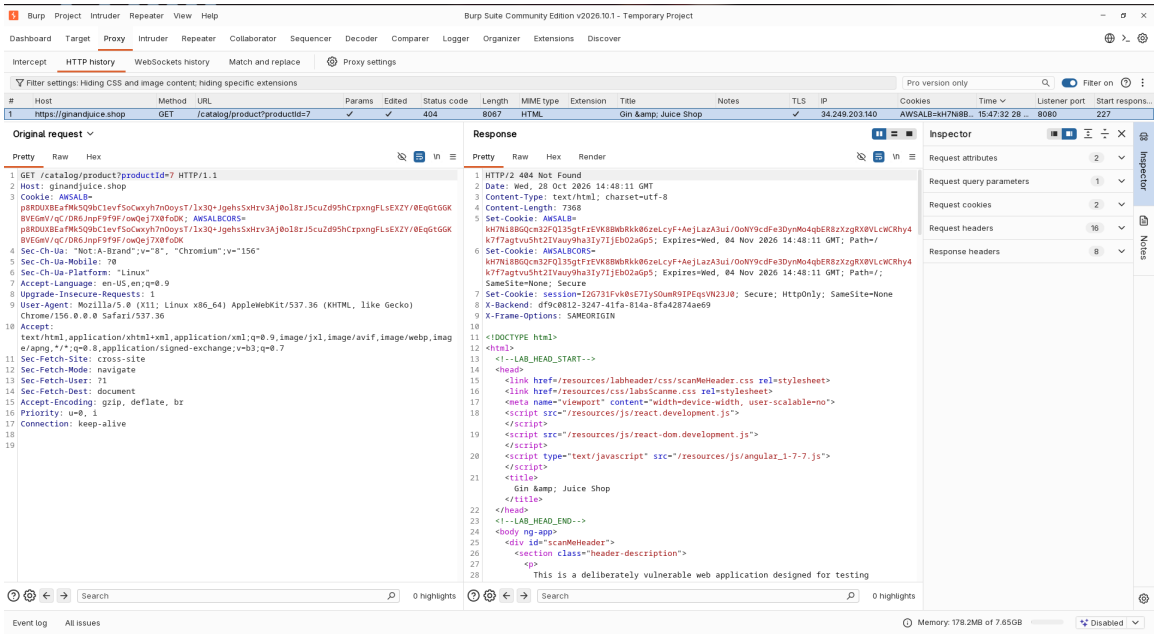


Figure 5: Requête avec un `productId` inexistant retournant une erreur 404 (historique HTTP de Burp).

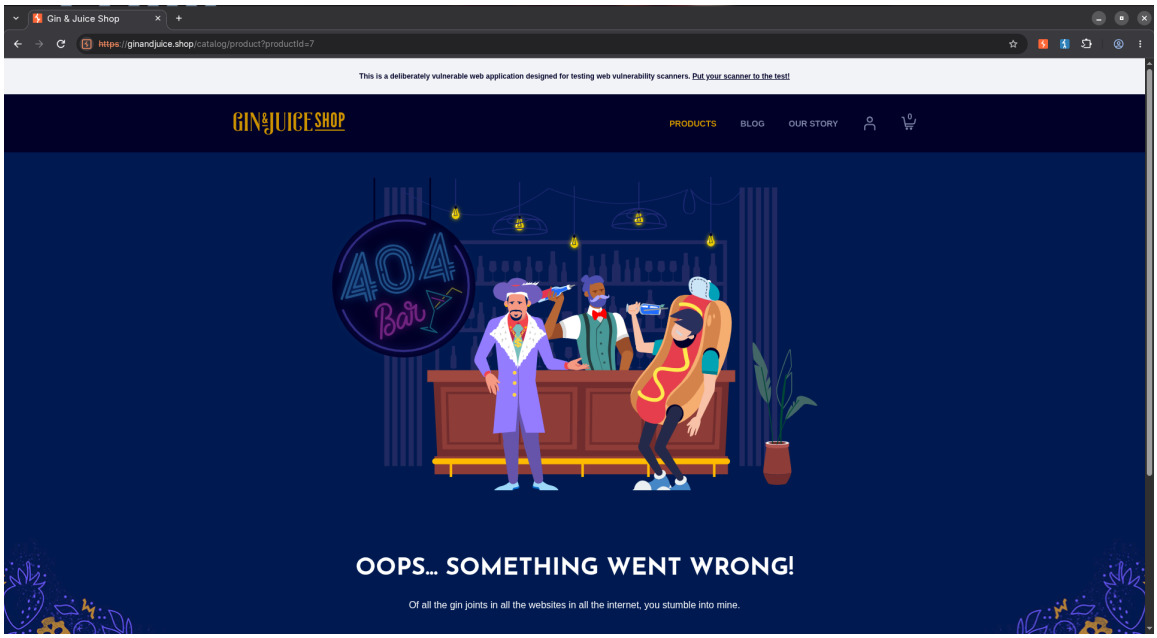


Figure 6: Rendu de la page d'erreur 404 dans le navigateur.

- **Code de statut HTTP :** `HTTP/2 404 Not Found`.
- **Corps de la réponse :** le serveur ne renvoie pas d'erreur interne non gérée (erreur 500), mais retourne une vue applicative d'erreur personnalisée titrée *« OOPS... SOMETHING WENT WRONG! »*, assortie du message : *« Of all the gin joints in all the websites in all the internet, you stumble into mine. »*.
- **Analyse :** le contrôleur a correctement exécuté la recherche en base de données, constaté l'absence d'enregistrement pour la clé demandée, et positionné le code de statut d'erreur standardisé (`404`) accompagné d'un gabarit HTML convivial.

3 Exercice 3 : Repeater — Rejouer et tester sans intermédiaire

L'outil Repeater permet de découpler les tests de l'interface graphique : une même requête peut être éditée, transmise et analysée de façon itérative pour observer les réponses du *backend*.

3.1 Tableau récapitulatif des tests sur le paramètre `productId`

Valeur testée	Code de statut	Description de la réponse
`1` ou `5`	`HTTP/2 200 OK`	Requête valide. Le serveur extrait l'enregistrement correspondant en base de données et renvoie la page produit complète.
`9999`	`HTTP/2 404 Not Found`	Ressource absente. La valeur est un entier valide mais aucune donnée n'y est rattachée. Renvoi d'un gabarit d'erreur 404 personnalisé.
`abc`	`HTTP/2 400 Bad Request`	Paramètre invalide. Le *backend* attend un entier ; la conversion de type (*cast*) échoue dans la couche de validation du *framework*, rejetant la requête avant l'interrogation SQL.
(vide)	`HTTP/2 400 Bad Request`	Paramètre manquant. L'URI `/catalog/product?productId=` viole la contrainte d'obligation du paramètre attendu par le contrôleur de routage.
`'` *(apostrophe)*	`HTTP/2 400 Bad Request`	Caractère de rupture SQL rejeté immédiatement par le filtre de typage applicatif sans déclencher d'exception interne (pas d'erreur 500).

Tableau 1: Réponses du serveur selon la valeur du paramètre `productId`.

3.2 Comportement face à un `User-Agent` fantaisiste et niveau de confiance

- **Comportement observé :** lors de l'envoi d'une requête HTTP valide comportant l'en-tête `User-Agent: Etudiant-3ACI/1.0`, le serveur ne lève aucune alerte et renvoie une réponse normale `HTTP/2 200 OK`.
- **Précision d'audit :** si un code `400 Bad Request` est survenu lors d'essais manuels, il s'explique par une malformation structurelle dans l'éditeur de texte de Repeater (altération des retours à la ligne `\r\n`, absence d'espace après les deux-points ou paramètre d'URL tronqué).
- **Conclusion sur la confiance :** le serveur **n'accorde aucune confiance de sécurité** à l'en-tête `User-Agent`. Cet en-tête est sous le contrôle exclusif de l'émetteur et peut être arbitrairement forgé pour simuler n'importe quel navigateur, moteur de recherche ou script. Un *backend* ne doit jamais fonder d'autorisation, de contrôle d'accès ou de décision critique sur cet en-tête.

3.3 Exploitation du rejeu de requêtes par un attaquant

- **Objectifs offensifs :**
  1. *Énumération méthodique (IDOR / BOLA) :* automatiser l'incrémentation d'identifiants séquentiels pour accéder à des ressources non protégées par un contrôle d'accès adéquat.
  2. *Fuzzing de sécurité :* injecter séquentiellement des caractères réservés (`'`, `"`, `--`, `/*`, `<script>`, `{{7*7}}`) pour tester les protections contre les injections SQL, XSS ou les vulnérabilités de gabarits (SSTI).
  3. *Force brute et* credential stuffing *:* tester des combinaisons de mots de passe ou des jetons de réinitialisation sans intervention manuelle dans un formulaire.

- **Exemple concret :** un auditeur identifie l'*endpoint* de consultation de facture `GET /api/v1/invoices?id=1024`. Si l'application vérifie que l'utilisateur est authentifié mais omet de vérifier que la facture 1024 lui appartient (vulnérabilité IDOR), rejouer la requête avec `id=1025`, `id=1026`, etc., permet à l'attaquant de récupérer les factures et données personnelles de l'ensemble des clients de la plateforme.

4 Exercice 4 : Intercepter et modifier une réponse HTTP

Cet exercice met en évidence la bidirectionnalité de l'interception : les données transmises par le serveur peuvent être altérées avant d'atteindre le moteur d'exécution du navigateur.

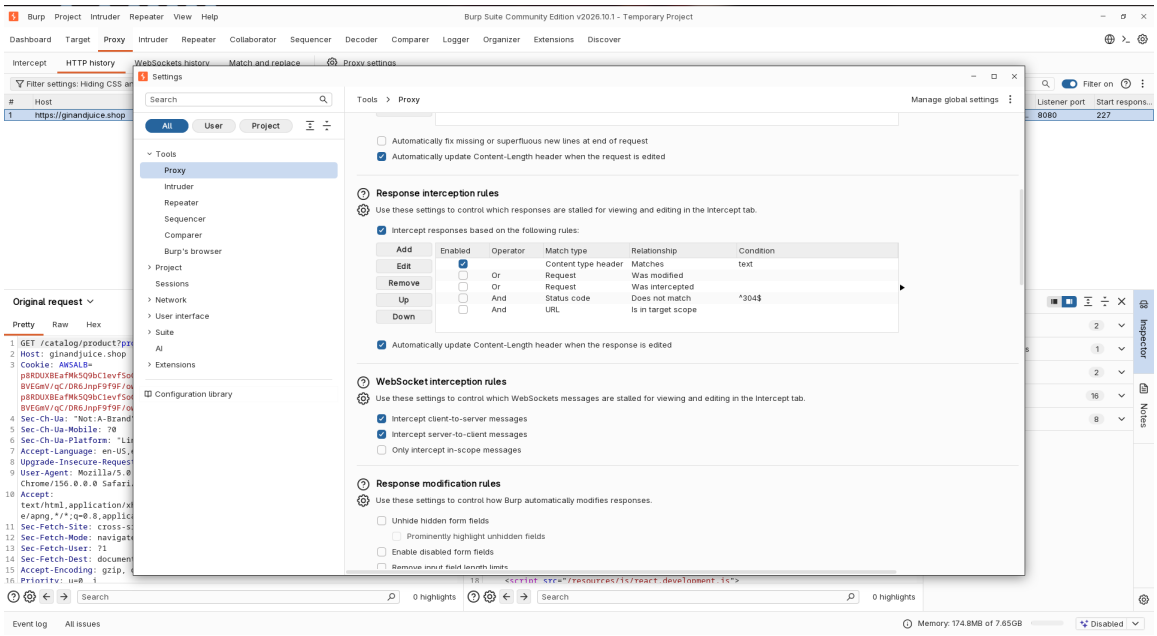


Figure 7: Activation de la règle d'interception des réponses HTTP dans les paramètres du proxy Burp Suite.

4.1 Affichage obtenu dans le navigateur

Le navigateur affiche **la version modifiée**. Après modification à la volée du code HTML intercepté dans Burp Suite (remplacement du titre ou du nom d'un produit par un identifiant personnalisé), le navigateur restitue visuellement la page modifiée sans aucune altération de comportement apparent.

4.2 Visibilité de la modification pour le serveur et le navigateur

- **Vis-à-vis du serveur :** le serveur ignore totalement la modification. Dès que les segments TCP ont quitté son interface réseau, il clôt son traitement transactionnel et n'a aucun moyen de superviser ce qui advient du flux en transit.

- **Vis-à-vis du navigateur :** le navigateur l'ignore également. Il reçoit un flux HTTP syntaxiquement conforme, transmis au travers d'un tunnel TLS valide négocié avec le proxy local. Dépourvu de signature cryptographique sur le contenu HTML applicatif, il analyse et rend le document reçu sans pouvoir détecter qu'il a été modifié après son émission par le serveur source.

4.3 Scénario d'attaque de type *Man-in-the-Middle* (MitM)

Si un attaquant contrôle un équipement d'interconnexion réseau (point d'accès Wi-Fi public compromis, routeur piraté, empoisonnement ARP) et parvient à contourner le chiffrement TLS (par exemple sur du trafic HTTP non chiffré ou via un certificat d'autorité frauduleux préalablement installé sur l'hôte), la capacité d'altérer les réponses lui permet de :

- **Injecter du code JavaScript malveillant (XSS en transit) :** intégrer des scripts pour dérober les jetons de session (`document.cookie`), enregistrer les frappes au clavier (*keylogger*) ou capturer les mots de passe saisis dans les formulaires.
- **Falsifier des informations critiques (fraude financière) :** modifier à la volée les coordonnées bancaires affichées (IBAN, montant de virement) pour rediriger des transactions légitimes vers le compte de l'attaquant.
- **Déployer des attaques de type *drive-by download* :** remplacer les liens de téléchargement de logiciels légitimes par des exécutables piégés ou injecter de fausses invites de mise à jour système incitant l'utilisateur à installer un logiciel malveillant.

5 Questions de synthèse

5.1 Interception HTTPS et modèle de confiance cryptographique

Tout le trafic du TP a été opéré en HTTPS, protocole conçu pour interdire l'écoute clandestine et la falsification de paquets via TLS. L'interception et la modification en clair par Burp Suite ont été rendues possibles grâce à un mécanisme de **terminaison TLS intermédiaire consentie** :

1. **Scission de la session TLS en deux tunnels indépendants :**
   - **Tunnel 1 (navigateur ↔ Burp Suite) :** le navigateur négocie sa connexion avec l'adresse locale du proxy (`127.0.0.1:8080`). Burp génère dynamiquement un certificat numérique pour le nom de domaine `ginandjuice.shop`, signé par l'autorité racine de certification locale de PortSwigger (*PortSwigger CA*).
   - **Tunnel 2 (Burp Suite ↔ serveur réel) :** Burp établit une session TLS standard avec le serveur distant en contrôlant la validité de son certificat officiel.
2. **Acceptation sans alerte de sécurité :** le navigateur embarqué accepte le certificat forgé par Burp car l'autorité racine de PortSwigger est préinstallée dans son magasin de certificats de confiance (*trust store*).
3. **Déchiffrement au point d'articulation :** Burp détenant les clés cryptographiques des deux sessions, les données transitent en clair au niveau applicatif à l'intérieur du proxy, permettant leur inspection et leur réécriture avant rechiffrement.

Le chiffrement TLS n'a donc pas été cassé algorithmiquement : c'est la racine de confiance du client qui a été configurée pour déléguer l'autorité de chiffrement au proxy local.

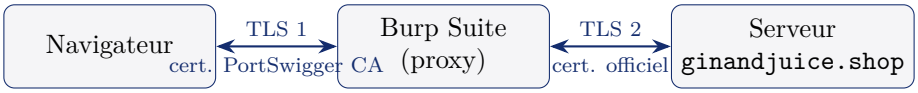


Figure 8: Terminaison TLS intermédiaire réalisée par Burp Suite.

5.2 Trois actions concrètes réalisables par un attaquant armé d'un proxy

Devant une application web dépourvue de protections robustes côté serveur, un attaquant peut exécuter :

1. **Le contournement des contrôles côté client :** remplacer des valeurs bloquées par l'interface utilisateur (champs `readonly`, `disabled`, attributs `maxlength` ou validations JavaScript), par exemple soumettre un prix unitaire nul ou négatif lors d'une commande.
2. **L'usurpation de ressources et l'escalade de privilèges (IDOR / *parameter tampering*) :** altérer des identifiants d'objets (`userId=10`, `role=admin`, `documentId=452`) dans les requêtes afin d'accéder aux données confidentielles d'autres comptes sans disposer des autorisations requises.
3. **L'injection de charges utiles non filtrées (SQLi / XSS) :** forger des requêtes contenant des métacaractères d'évasion SQL ou des charges scriptées sans être entravé par les règles d'encodage du formulaire HTML, en analysant la structure brute des réponses d'erreur pour cartographier le *backend*.

5.3 Réfutation de l'argument : « De toute façon ce champ est en lecture seule dans le formulaire, personne ne peut le modifier »

Cette assertion repose sur une faille conceptuelle majeure : **confondre l'ergonomie de l'interface utilisateur avec la sécurité du système d'information.**

Les directives HTML telles que `readonly`, `disabled` ou les vérifications écrites en JavaScript s'exécutent intégralement sur le terminal de l'utilisateur. Elles n'ont de valeur que pour guider un utilisateur bienveillant et prévenir les erreurs de saisie.

Un attaquant ne dépend aucunement du navigateur ni du formulaire d'origine. Grâce à un proxy d'interception ou à un utilitaire en ligne de commande (`curl`), la requête HTTP peut être interceptée après son émission ou construite de toutes pièces. Le champ en lecture seule peut ainsi être librement altéré, injecté ou supprimé avant que les octets n'atteignent le serveur.

Le principe cardinal du développement sécurisé est : « **Ne jamais faire confiance aux données transmises par le client** » (*Never trust client input*). Toute contrainte d'intégrité, de prix ou d'autorisation doit impérativement être vérifiée et imposée de manière stricte côté serveur.

5.4 Impact direct sur les habitudes de développement logiciel

- **Emplacement du code concerné :** la couche de validation des contrôleurs *backend* et les gestionnaires de services métier (*request validators*, *Data Transfer Objects*, *authorization gates*).
- **Évolution de la pratique de développement :**
  - *Pratique vulnérable (avant le TP) :* récupérer directement les valeurs sensibles soumises par le corps de requête ou l'URL (prix d'un produit, montant de remise, identifiant de propriétaire `owner_id`) en supposant que l'interface client a transmis des valeurs saines et verrouillées.
  - *Pratique sécurisée (après le TP) :*
    1. **Recalcul côté serveur :** ne jamais accepter de prix ou de paramètre financier provenant du client. Le contrôleur doit uniquement recevoir l'identifiant du produit (`productId`) et interroger la base de données de référence pour en extraire le tarif officiel.

2. **Contrôle d'accès rigoureux sur chaque ressource :** pour tout paramètre identifiant reçu (`productId`, `orderId`, `profileId`), vérifier systématiquement en base de données que l'utilisateur rattaché au jeton de session possède effectivement les privilèges nécessaires pour consulter ou modifier l'enregistrement ciblé.

Conclusion

Ce TP a permis de maîtriser les fonctions de base de Burp Suite : observation du trafic, interception et modification des requêtes et des réponses, et rejeu de requêtes avec Repeater. Il met en évidence un principe fondamental : le client est sous le contrôle de l'utilisateur, donc toute validation et tout contrôle d'accès doivent être appliqués côté serveur.